

Guia completa de iconos

Este documento resume la configuracion y el uso de iconos en el proyecto con el enfoque actual basado en `EAMIIcons` y en la lista central `ExternalIcons` .

Objetivo

Generalizar la configuracion de iconos externos para que no dependa de cada webpart y pueda reutilizarse en modulos como:

- `Tiles`
- `BoardExplorer`
- otros modulos futuros

El render de iconos queda centralizado en `EAMIIcons` .

Modelo general

Se usa una lista central de SharePoint:

- `ExternalIcons`

Cada registro representa una libreria de iconos externa.

La webpart no intenta adivinar el formato. Se le indica explicitamente mediante la columna `Type` .

Cada webpart puede:

- activar o no el uso de iconos externos
 - usar todas las librerias de su `Section`
 - o limitarse a algunas librerias concretas
-

Lista `ExternalIcons`

Campos recomendados

1. `Title`

- Tipo: `Single line of text`
- Uso: nombre visible de la libreria
- Ejemplos:
 - `Lucide`
 - `Heroicons`
 - `CustomIconsSvg`

2. `Section`

- Tipo: `Choice`
- Uso: para que modulo aplica
- Ejemplos:
 - `Tiles`
 - `BoardExplorer`
- Nota:
 - puede venir vacio o `null`
 - si viene vacio, la libreria se considera global

3. `Type`

- Tipo: `Choice`
- Uso: formato/origen de la libreria
- Valores soportados:
 - `JSON catalog`
 - `Metadata endpoint`
 - `SharePoint folder`
 - `Inline SVG`
- Nota:
 - este campo si es obligatorio

4. `Source`

- Tipo: `Single line of text`
- Uso: origen de la libreria
- Puede ser:
 - URL a JSON
 - endpoint `?meta`
 - carpeta SharePoint

- URL a JSON con SVG inline

5. SortOrder

- Tipo: Number
- Uso: prioridad entre librerías
- Regla:
 - menor valor = mayor prioridad

6. IsEnabled

- Tipo: Yes/No
- Uso: activar o desactivar la librería sin borrarla

Tipos soportados

1. JSON catalog

JSON con `baseUrl` + `icons`.

Ejemplo:

```
{
  "library": "MiLibrería",
  "baseUrl": "https://unpkg.com/lucide-static@0.562.0/icons/",
  "icons": [
    "home",
    "user",
    "users",
    "settings",
    "search",
    "bell",
    "calendar",
    "clock",
    "mail",
    "phone"
  ]
}
```

Resultado esperado:

- construye URLs como:
 - `https://unpkg.com/lucide-static@0.562.0/icons/home.svg`
 - `https://unpkg.com/lucide-static@0.562.0/icons/user.svg`
 - `https://unpkg.com/lucide-static@0.562.0/icons/settings.svg`

2. Metadata endpoint

Endpoint tipo `?meta`.

Ejemplo real:

```
https://unpkg.com/lucide-static@0.562.0/icons/?meta
```

También puede usarse la carpeta base:

```
https://unpkg.com/lucide-static@0.562.0/icons/
```

Respuesta típica:

```
{
  "package": "lucide-static",
  "version": "0.562.0",
  "prefix": "/icons/",
  "files": [
    {
      "path": "/icons/home.svg",
      "size": 459,
      "type": "image/svg+xml"
    },
    {
      "path": "/icons/user.svg",
      "size": 375,
      "type": "image/svg+xml"
    },
    {
      "path": "/icons/settings.svg",
      "size": 666,
      "type": "image/svg+xml"
    }
  ]
}
```

Resultado esperado:

- detecta los `.svg` del array `files[]`
- arma el catalogo automaticamente

3. SharePoint folder

Carpeta SharePoint con archivos `.svg`.

Ejemplo con URL absoluta:

```
https://raonacloud.sharepoint.com/sites/Connect/SiteAssets/Icons/Corporate
```

Ejemplo con URL relativa:

```
/sites/Connect/SiteAssets/Icons/Corporate
```

Contenido esperado en la carpeta:

```
home.svg
user.svg
settings.svg
search.svg
mail.svg
phone.svg
```

Resultado esperado:

- usa el nombre del archivo sin `.svg` como nombre del icono
- ejemplo:
 - `home.svg` -> `home`
 - `user.svg` -> `user`
 - `settings.svg` -> `settings`

4. Inline SVG

JSON simple donde cada propiedad apunta a un SVG inline.

Ejemplo:

```
{
  "icon1": "<svg xmlns=\"http://www.w3.org/2000/svg\" width=\"24\" height=\"24\" fill=\"none\" viewBox=\"0 0 24 24\"><path stroke=\"
  \"icon2\": \"<svg xmlns=\"http://www.w3.org/2000/svg\" width=\"24\" height=\"24\" fill=\"none\" viewBox=\"0 0 24 24\"><path stroke=\"
  \"icon3\": \"<svg xmlns=\"http://www.w3.org/2000/svg\" width=\"24\" height=\"24\" fill=\"none\" viewBox=\"0 0 24 24\"><circle cx=\"12
}
```

Resultado esperado:

- cada propiedad es el nombre del icono
- el valor es el SVG inline
- no necesita `baseUr1`
- no necesita archivos `.svg` separados

Ejemplos reales de registros en `ExternalIcons`

Lucide

- Title: Lucide
- Section: Tiles
- Type: Metadata endpoint
- Source: `https://unpkg.com/lucide-static@0.562.0/icons/?meta`
- SortOrder: 1
- IsEnabled: Yes

Heroicons

- Title: Heroicons
- Section: Tiles
- Type: Metadata endpoint
- Source: `https://unpkg.com/heroicons@2.1.1/24/outline/?meta`
- SortOrder: 2
- IsEnabled: Yes

Carpeta SharePoint

- Title: CorporateIcons
- Section: Tiles
- Type: SharePoint folder
- Source: `/sites/Connect/SiteAssets/Icons/Corporate`
- SortOrder: 3
- IsEnabled: Yes

JSON con lista

- Title: CustomIcons
- Section: Tiles
- Type: JSON catalog
- Source: `https://.../customicons.json`
- SortOrder: 4
- IsEnabled: Yes

JSON con SVG inline

- Title: CustomIconsSvg
- Section: Tiles
- Type: Inline SVG
- Source: `https://.../customicons1.json`
- SortOrder: 5
- IsEnabled: Yes

Regla de prioridad

Si dos librerías tienen un icono con el mismo nombre:

- gana la librería con menor `SortOrder`

Ejemplo:

- `Lucide -> SortOrder = 1`
- `Heroicons -> SortOrder = 2`

Si ambas tienen `home`, gana `Lucide`.

Esto aplica para:

- deduplicacion
- busquedas globales
- prioridades internas

En los tabs del selector, cada libreria sigue mostrando sus propios iconos.

Configuracion por webpart

Cada webpart mantiene:

1. `EnableExternalIconLibraries`
 - toggle para activar o no la funcionalidad
2. `ExternalIconLibrariesFilterText`
 - textarea para limitar librerias concretas por nombre
 - formato: nombres separados por coma

Ejemplos validos:

```
Lucide
```

```
Lucide, Heroicons
```

```
Lucide, Heroicons, CustomIconsSvg
```

Importante:

- los nombres deben coincidir con el campo `Title` de la lista `ExternalIcons`

Regla de uso

1. Si el toggle esta apagado:
 - no usa iconos externos
2. Si el toggle esta encendido y el textarea esta vacio:
 - usa todas las librerias activas de la `Section`
3. Si el toggle esta encendido y el textarea tiene valores:
 - usa solo las librerias indicadas

Regla de `Section`

- si `Section` tiene valor, aplica a esa seccion
- si `Section` viene vacio o `null`, la libreria es global

Formato correcto para llamar un icono externo

Si se quiere usar un icono externo de forma explicita, el valor correcto es:

```
ext:<LibraryName>:<IconName>
```

Ejemplos:

- `ext:Lucide:home`
- `ext:Heroicons:academic-cap`
- `ext:CustomIconsSvg:icon1`

Ejemplo correcto

```
<EAMIIcons
  icon="ext:Lucide:home"
  isFluentUI={true}
  isCustomSVGLoaded={true}
/>
```

Importante

Esto no garantiza usar una libreria externa concreta:

```
<EAMIIcons icon="home" />
```

Si se pone solo `home`, el sistema no asume automaticamente que quieres el `home` de Lucide.

Si quieres una libreria externa concreta, hay que usar siempre:

```
ext:<LibraryName>:<IconName>
```

Ejemplo combinado

Se pueden combinar varias librerias en la misma configuracion.

Ejemplo:

- Lucide
- Heroicons
- CorporateIcons
- CustomIconsSvg

Resultado funcional esperado:

- cada libreria aparece en su tab correspondiente
- la busqueda funciona sobre todas las librerias cargadas
- el valor guardado sigue siendo `ext:<LibraryName>:<IconName>`

Ejemplos:

- `ext:Lucide:home`
- `ext:Heroicons:academic-cap`
- `ext:CustomIconsSvg:icon1`

Librerias CDN utiles

Listado de librerias de iconos accesibles por UNPKG con soporte para `?meta`.

Lucide

```
https://unpkg.com/lucide-static@latest/icons/
```

Heroicons outline

```
https://unpkg.com/heroicons@2.1.1/24/outline/?meta
```

Heroicons solid

```
https://unpkg.com/heroicons@2.1.1/24/solid/?meta
```

Tabler Icons

```
https://unpkg.com/@tabler/icons@latest/icons/?meta
```

Bootstrap Icons

```
https://unpkg.com/bootstrap-icons@latest/icons/?meta
```

Feather Icons

```
https://unpkg.com/feather-icons@latest/dist/icons/?meta
```

Phosphor Icons

```
https://unpkg.com/@phosphor-icons/core@latest/assets/?meta
```

Remix Icons

```
https://unpkg.com/remixicon@latest/icons/?meta
```

Ionicons

```
https://unpkg.com/ionicons@latest/dist/ionicons/svg/?meta
```

Simple Icons

```
https://unpkg.com/simple-icons@latest/icons/?meta
```

Bonus

Radix Icons

```
https://unpkg.com/@radix-ui/icons@latest/dist/svg/?meta
```

CSS.gg

```
https://unpkg.com/css.gg@latest/icons/svg/?meta
```

Octicons

```
https://unpkg.com/@primer/octicons@latest/build/svg/?meta
```

Notas sobre `?meta`

- `?meta` devuelve un JSON con todos los archivos disponibles
- es útil para:
 - pickers de iconos
 - autocompletado
 - carga dinámica

Override con `Custom SVG Url`

`Custom SVG Url` puede sobrescribir iconos si el JSON contiene una key con el mismo nombre del icono.

Ejemplo:

```
{
  "home": "<svg ...></svg>",
  "search": "<svg ...></svg>"
}
```

Si el icono seleccionado es:

```
ext:Lucide:home
```

el sistema puede usar el SVG definido en `home` como override.

Resultado esperado

Con este enfoque:

- la administracion queda centralizada
- las webparts pueden reutilizar librerias
- cada webpart puede restringir cuales usa
- el sistema deja de adivinar formatos
- la prioridad entre duplicados queda controlada con `SortOrder`
- escala mejor a otras secciones
- el render de iconos queda centralizado en `EAMIIcons`

Decisiones cerradas

1. Lista central: `ExternalIcons`
2. Columna `Section`
3. Columna `Type`
4. Tipos finales:
 - `JSON catalog`
 - `Metadata endpoint`
 - `SharePoint folder`
 - `Inline SVG`
5. Columna `Source`
6. Columna `SortOrder`
7. Columna `IsEnabled`
8. Prioridad por menor `SortOrder`
9. En cada webpart:
 - `toggle`
 - textarea para filtrar librerias por nombre
10. Si no se selecciona ninguna libreria en la webpart:
 - se usan todas las de la `Section`
11. Si `Section` viene vacio:
 - la libreria es global
12. Para usar una libreria externa concreta:
 - hay que usar `ext:<LibraryName>:<IconName>`

Override de iconos con Custom SVG Url

Este documento explica como seguir sobrescribiendo iconos usando `Custom SVG Url` .

Se puede seguir reemplazando iconos

Si.

Se pueden seguir reemplazando iconos mediante el JSON cargado desde `Custom SVG Url` .

Esto aplica a:

- iconos internos del proyecto
- iconos registrados en `IconsResource.json`
- iconos usados por nombre
- iconos externos cuando se usa la key correcta

Como funciona

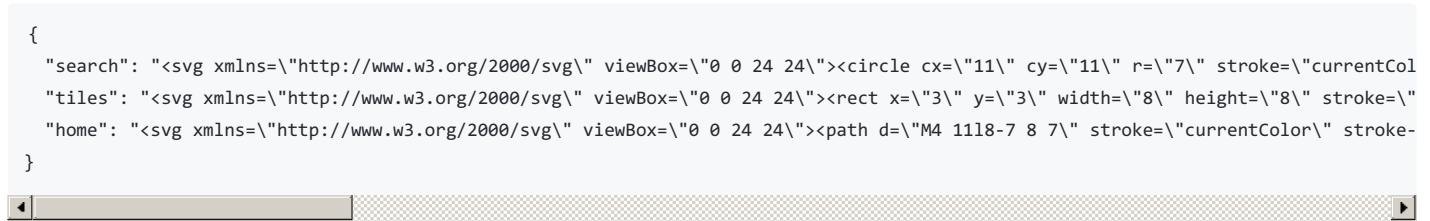
`Custom SVG Url` carga un JSON con pares:

- `key` -> nombre del icono
- `value` -> SVG inline

Si el nombre coincide con el icono que se esta intentando renderizar, ese SVG sobrescribe al original.

Formato del JSON

Ejemplo:



Caso 1. Reemplazar un icono interno

Si el proyecto usa en código:



Y en Custom SVG Url existe:



Entonces el icono `search` queda sobrescrito por ese SVG.

Ejemplos típicos de iconos internos que pueden sobrescribirse

- `search`
- `tiles`
- `collapse`
- `uncollapse`
- `notificationsEmpty`
- `FavoriteStar`
- `FavoriteStarFill`

Caso 2. Reemplazar un icono externo

Si el valor guardado del icono es:



Y en Custom SVG Url existe esta key:



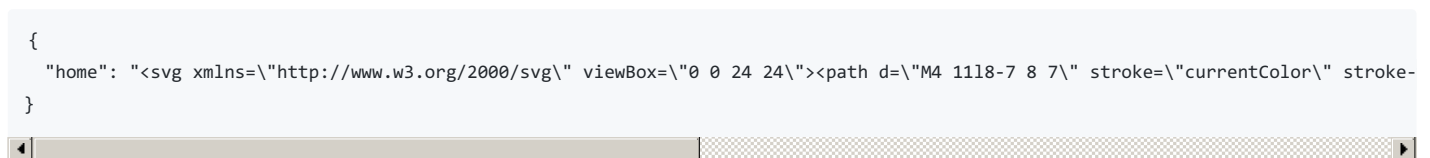
Entonces el sistema puede usar ese SVG como override del `home` externo.

Ejemplo completo

Icono seleccionado:



JSON en Custom SVG Url :



Resultado:

- el render usa el SVG de `home` definido en Custom SVG Url

- en vez del `home` original de Lucide

Regla practica

Si quieres sobrescribir un icono:

1. mira el nombre real del icono que se esta usando
2. usa exactamente ese nombre como key en el JSON
3. pon el SVG inline como valor

Ejemplo correcto de key

Si el codigo usa

```
<EAMIIcons icon="tiles" />
```

La key correcta es:

```
{
  "tiles": "<svg ...></svg>"
}
```

Si el valor guardado es

```
ext:Heroicons:academic-cap
```

La key correcta es:

```
{
  "academic-cap": "<svg ...></svg>"
}
```

Resumen

- si, se puede seguir sobrescribiendo con `Custom SVG Url`
- funciona con iconos internos y externos
- para externos, la key a usar es el nombre real del icono
- no hace falta poner `ext:Lucide:home` como key
- basta con poner `home` si el icono externo seleccionado es `ext:Lucide:home`